DSA

Credit Hours: 3-1

Course Outline

Introduction to data structure and algorithm

16-Week Course Plan

Week 1

Week 1: Introduction to Data Structures and Algorithms (3-1)

Lecture 1: Introduction to Data Structures and Algorithms (1.5 hours)

Data Structures and Algorithms

Learning Objectives

* Define data structures and algorithms

* Identify the importance of data structures and algorithms in computer science

Concepts

Data structures and algorithms are the building blocks of computer science. Data structures are the way we organize and store data, while algorithms are the procedures for solving problems. Understanding data structures and algorithms is crucial for developing efficient and effective software.

Lecture 2: Types of Data Structures (1.5 hours)

Data Structures

Learning Objectives

* Identify the main types of data structures

* Explain the characteristics of each data structure

Concepts

There are several types of data structures, including:

* Arrays: a collection of elements of the same data type stored in contiguous memory locations

* Linked Lists: a dynamic collection of elements where each element points to the next element

* Stacks and Queues: data structures that follow the Last-In-First-Out (LIFO) and First-In-First-Out (FIFO) principles respectively

Lecture 3: Introduction to Algorithm Design (1 hour)

Algorithm Design

Learning Objectives

* Define algorithm design

* Explain the steps involved in algorithm design

Concepts

Algorithm design involves breaking down a problem into smaller sub-problems, identifying the input and output, and developing a step-by-step procedure to solve the problem. The steps involved in algorithm design include:

* Problem definition

* Input and output identification

* Algorithm specification

* Algorithm implementation

Lab 1: Introduction to Data Structures and Algorithms Lab (1 hour)

Lab 1: Introduction to Data Structures and Algorithms

Learning Objectives

* Understand the basics of data structures and algorithms

* Implement a simple data structure using a programming language

Concepts

In this lab, students will implement a simple array data structure using a programming language such as Python or Java. They will learn how to create, manipulate, and access array elements.

Week 2

Week 2: Arrays and Linked Lists

3-1

Lecture 1: Arrays (1.5 hours)

Learning Objectives

* Understand the concept of arrays

* Learn how to declare and initialize arrays

* Understand the basic operations on arrays (access, update, delete)

Concepts

Arrays are a fundamental data structure in programming, allowing for efficient storage and manipulation of collections of elements. In this lecture, we will cover the basics of arrays, including declaration, initialization, and basic operations.

* Declaration: `type name[size];` (e.g., `int arr[5];`)

* Initialization: `type name[size] = {value1, value2, ..., valueN};` (e.g., `int arr[5] = {1, 2, 3, 4, 5};`)

* Access: `name[index]` (e.g., `arr[3]`)

* Update: `name[index] = value;` (e.g., `arr[3] = 10;`)

* Delete: `delete[] name;` (e.g., `delete[] arr;`)

Lecture 2: Linked Lists (1.5 hours)

Learning Objectives

* Understand the concept of linked lists

* Learn how to implement a basic singly linked list

* Understand the basic operations on linked lists (insert, delete, search)

Concepts

Linked lists are a dynamic data structure in which elements are linked together using pointers. In this lecture, we will cover the basics of linked lists, including implementation and basic operations.

* Node structure: `struct Node {int data; Node* next;};`

* Insertion: `Node* insert(Node* head, int value);`

* Deletion: `Node* delete(Node* head, int value);`

* Search: `Node* search(Node* head, int value);`

Lecture 3: Array vs. Linked List (1 hour)

Learning Objectives

* Compare and contrast arrays and linked lists

* Understand the trade-offs between arrays and linked lists

Concepts

In this lecture, we will compare and contrast arrays and linked lists, highlighting the trade-offs between these two data structures.

* Space complexity: arrays vs. linked lists

* Time complexity: arrays vs. linked lists

* Insertion and deletion: arrays vs. linked lists

Lab 1: Array and Linked List Implementations (1 hour)

Learning Objectives

* Implement basic array and linked list operations

* Test and debug array and linked list implementations

Concepts

In this lab, students will implement basic array and linked list operations and test and debug their implementations.

Week 3

Week 3: Arrays and Linked Lists (3-1)

Lecture 1: Arrays (1.5 hours)

Arrays

Arrays are a fundamental data structure in computer science. An array is a collection of elements of the same data type stored in contiguous memory locations.

Learning Objectives

* Define an array and its elements

* Explain the concept of indexing in arrays

* Identify the advantages and disadvantages of arrays

Concepts

* Array declaration and initialization

* Array indexing and accessing elements

* Array operations (e.g., traversal, searching, sorting)

Lecture 2: Array Operations (1.5 hours)

Array Operations

Array operations include traversal, searching, sorting, and inserting/deleting elements. Understanding these operations is crucial for efficient array management.

Learning Objectives

* Explain the concept of array traversal

* Describe the algorithms for searching and sorting arrays

* Identify the time and space complexities of array operations

Concepts

* Array traversal algorithms (e.g., linear search, binary search)

* Array sorting algorithms (e.g., bubble sort, selection sort)

* Time and space complexities of array operations

Lecture 3: Linked Lists (1 hour)

Linked Lists

A linked list is a dynamic data structure where each element (node) points to the next node. Linked lists are useful for implementing stacks, queues, and other data structures.

Learning Objectives

* Define a linked list and its nodes

* Explain the concept of node traversal in linked lists

* Identify the advantages and disadvantages of linked lists

Concepts

* Linked list node structure and operations

* Node traversal algorithms (e.g., recursive, iterative)

* Linked list advantages and disadvantages

Lab 1: Array and Linked List Implementation (1 hour)

Lab Exercise

Implement an array and a linked list in a programming language of your choice. Perform basic operations such as insertion, deletion, and traversal. Analyze the time and space complexities of these operations.

Week 4

Week 4: Arrays and Linked Lists

Lecture 1: Introduction to Arrays (45 minutes)

Arrays

Arrays are a fundamental data structure in computer science.

Learning Objectives

• Define an array and its applications.

• Understand the concept of indexing in arrays.

• Identify the advantages and disadvantages of arrays.

Arrays are a collection of elements of the same data type stored in contiguous memory locations.

Key Concepts

• Array declaration and initialization.

• Array indexing and accessing elements.

• Array operations: insertion, deletion, and search.

Lecture 2: Array Operations (45 minutes)

Array Operations

Arrays support various operations, including insertion, deletion, and search.

Learning Objectives

• Understand the array insertion operation.

• Identify the different methods for deleting elements from an array.

• Explain the array search operation.

Array operations are performed using loops, conditional statements, and functions.

Key Concepts

• Insertion: shifting elements to accommodate new elements.

• Deletion: shifting elements to fill the gap left by the deleted element.

• Search: using loops to locate a specific element.

Lecture 3: Linked Lists (45 minutes)

Linked Lists

Linked lists are a dynamic data structure consisting of nodes connected by pointers.

Learning Objectives

• Define a linked list and its applications.

• Understand the concept of node and pointer.

• Identify the advantages and disadvantages of linked lists.

Linked lists are suitable for insertion and deletion operations at arbitrary positions.

Key Concepts

• Node structure and pointer implementation.

• Linked list operations: insertion, deletion, and search.

Lab 1: Array and Linked List Implementation (120 minutes)

• Implement an array using a programming language of choice.

• Implement a linked list with insertion, deletion, and search operations.

• Compare and contrast arrays and linked lists.

Week 5

Week 5: **Hash Tables**

3-1

Lecture 1: Introduction to Hash Tables

Learning Objectives

• Understand the definition and basics of hash tables

• Learn the advantages and disadvantages of hash tables

Concepts

A hash table is a data structure that stores key-value pairs in an array using a hash function to map keys to indices of the array. The key is the unique identifier for each value, and the value is the actual data. Hash tables provide fast lookups, insertions, and deletions with an average time

complexity of O(1). However, they can be prone to collisions, which can decrease performance.

Lecture 2: Hash Function and Collision Resolution

Learning Objectives

• Understand the importance of a good hash function

• Learn collision resolution techniques

Concepts

A good hash function distributes keys evenly across the array to minimize collisions. Collision resolution techniques include open addressing, where the next slot in the array is searched, and separate chaining, where a linked list is used to store colliding key-value pairs.

Lecture 3: Hash Table Operations

Learning Objectives

• Understand how to implement basic hash table operations

• Learn about resizing and dynamic arrays

Concepts

Hash table operations include insertions, deletions, search, and traversal. Resizing is necessary to maintain performance when the load factor exceeds a certain threshold. This is achieved by creating a new array and rehashing all key-value pairs.

Lab 1: Implementing a Basic Hash Table

Learning Objectives

• Implement a basic hash table using a hash function and collision resolution technique

• Test the hash table with various operations

Instructions

Implement a simple hash table using C++ or Python, including a hash function and collision resolution. Test the hash table with insertion, deletion, and search operations.

Week 6

Week 6: Sorting Algorithms

Lecture 1: Introduction to Sorting Algorithms (45 minutes)

Learning Objectives

• Understand the need for sorting algorithms

• Identify different types of sorting algorithms

• Compare the time and space complexity of various sorting algorithms

Concepts

Sorting algorithms are used to arrange elements of an array in a specific order, either ascending or descending. There are two main types: comparison-based and non-comparison-based sorting algorithms. Comparison-based algorithms sort by comparing elements, while non-comparison-based algorithms sort by rearranging elements without comparisons. Some common comparison-based algorithms include Bubble Sort, Selection Sort, and Insertion Sort. Non-comparison-based algorithms include Radix Sort and Counting Sort.

Lecture 2: Bubble Sort and Selection Sort (45 minutes)

Learning Objectives

• Understand the Bubble Sort algorithm

• Understand the Selection Sort algorithm

• Compare the time and space complexity of Bubble Sort and Selection Sort

Concepts

Bubble Sort works by repeatedly swapping adjacent elements if they are in the wrong order. Selection Sort works by repeatedly finding the minimum element from the unsorted part and putting it at the beginning. Both algorithms have a time complexity of O(n^2) and are not suitable for large datasets.

Lecture 3: Insertion Sort and Radix Sort (45 minutes)

Learning Objectives

• Understand the Insertion Sort algorithm

• Understand the Radix Sort algorithm

• Compare the time and space complexity of Insertion Sort and Radix Sort

Concepts

Insertion Sort works by inserting elements into a sorted subarray one at a time. Radix Sort works by sorting numbers digit by digit from least significant digit to most significant digit. Insertion Sort has a time complexity of O(n^2) while Radix Sort has a time complexity of O(nk), where n is the number of elements and k is the number of digits in the radix sort.

Lab 1: Implementing Bubble Sort and Insertion Sort (60 minutes)

Learning Objectives

• Implement Bubble Sort and Insertion Sort algorithms in a programming language of choice

• Compare the performance of Bubble Sort and Insertion Sort on a sample dataset

Concepts

Implement the Bubble Sort and Insertion Sort algorithms in a programming language of choice, and compare their performance on a sample dataset.

Week 7

Week 7: Hash Tables and Graphs

Lecture 7-1: Hash Tables

Learning Objectives

* Understand the concept of hash tables and their applications

* Learn how to implement a basic hash table in code

* Understand the trade-offs of hash tables in terms of time and space complexity

Concepts

A hash table is a data structure that stores key-value pairs in an array using a hash function to map keys to indices of the array. The hash function takes a key as input and generates an index that corresponds to the location where the value associated with that key should be stored. Hash tables provide efficient search, insertion, and deletion operations with an average time complexity of O(1).

Lecture 7-2: Hash Table Implementation

Learning Objectives

* Learn how to implement a basic hash table using arrays and separate chaining

* Understand the concept of collision resolution using chaining

* Learn how to handle collisions in a hash table

Concepts

To implement a basic hash table, we can use arrays and separate chaining to store key-value pairs. When a collision occurs, we can use a linked list to store the colliding elements. We need to handle collisions using techniques such as linear probing or quadratic probing.

Lecture 7-3: Graphs

Learning Objectives

* Understand the concept of graphs and their applications

* Learn how to represent graphs using adjacency matrices and adjacency lists

* Understand the time and space complexity of graph traversal algorithms

Concepts

A graph is a non-linear data structure consisting of vertices and edges that connect them. Graphs can be represented using adjacency matrices or adjacency lists. Adjacency matrices are suitable for dense graphs, while adjacency lists are more efficient for sparse graphs. Graph traversal algorithms such as BFS and DFS can be used to traverse the graph and perform operations such as finding the shortest path or detecting cycles.

Lab 7-0: Implementing a Hash Table and Graph Traversal

Tasks

* Implement a basic hash table using arrays and separate chaining

* Implement graph traversal algorithms (BFS and DFS)

* Use the graph traversal algorithms to find the shortest path between two vertices in a graph

Week 8

Week 8: Data Structures - Heaps and Priority Queues

Lecture 1: Introduction to Heaps (45 minutes)

Heaps and Priority Queues

Heaps are specialized tree-based data structures that satisfy the heap property. A heap is a complete binary tree, where each parent node is either greater than (max heap) or less than (min heap) its children. Heaps are used to implement priority queues.

Concepts

* Definition of a heap

* Types of heaps: max heap and min heap

* Heap property

Lecture 2: Heap Properties and Operations (45 minutes)

Heap Properties and Operations

* Heap ordering: max heap and min heap

* Heap operations:

+ Insertion

+ Deletion

+ Heapify-up

+ Heapify-down

Lecture 3: Implementation of Heaps (45 minutes)

Implementation of Heaps

* Array representation of a heap

* Building a heap from an array

* Heap operations implementation

Lab 1: Heap Implementation in Python (60 minutes)

* Implement a max heap and a min heap in Python

* Implement heap operations (insertion, deletion, heapify-up, heapify-down)

Note: The lab will be based on the concepts learned in the lectures. Students will implement heap data structures and operations in a programming language of their choice.

Week 9

Week 9: Advanced Data Structures (3-1)

Lecture 1: Hash Tables (60 minutes)

Learning Objectives

* Understand the concept of hash tables

* Learn to implement basic hash table operations

Concepts

Hash tables store key-value pairs using a hash function. The hash function maps keys to indices of a backing array. Hash tables provide fast lookups, insertions, and deletions. Common operations include `insert(key, value)`, `get(key)`, and `remove(key)`. Implementations include separate chaining and open addressing.

Lecture 2: Hash Table Operations (60 minutes)

Learning Objectives

* Learn to handle collisions in hash tables

* Understand the trade-offs between different hash table implementations

Concepts

Collision resolution techniques include chaining and open addressing. Chaining uses a linked list at each index, while open addressing uses probing sequences. Hash table performance depends on the quality of the hash function and the chosen collision resolution technique.

Lecture 3: Advanced Hash Table Topics (60 minutes)

Learning Objectives

* Learn about dynamic resizing and load factors

* Understand the implications of hash table size on performance

Concepts

Hash tables can dynamically resize to maintain a desired load factor. The load factor is the ratio of filled slots to total slots. A load factor close to 1 can lead to poor performance due to increased collision rates. Strategies for dynamic resizing include doubling and discarding.

Lab 1: Implementing a Hash Table (120 minutes)

Learning Objectives

* Implement a basic hash table using separate chaining

* Experiment with different hash table sizes and collision resolution techniques

Tasks

* Implement a `HashTable` class with basic operations (`insert`, `get`, `remove`)

* Experiment with different hash functions and collision resolution techniques

* Measure and analyze the performance of the hash table under different loads

Week 10

Week 10: Graph Algorithms (3-1)

Lecture 1: Graph Traversal Algorithms

Learning Objectives

* Understand the different types of graph traversal algorithms

* Learn how to implement Depth-First Search (DFS) and Breadth-First Search (BFS) algorithms

Concepts

Graph traversal algorithms are used to visit and process all nodes in a graph. DFS and BFS are two common types of graph traversal algorithms. DFS visits nodes in a graph by exploring as far as possible along each branch before backtracking. BFS visits nodes in a graph by exploring all nodes at a given depth level before moving on to the next level.

Lecture 2: Minimum Spanning Tree (MST) Algorithms

Learning Objectives

* Understand the concept of Minimum Spanning Tree (MST)

* Learn how to implement Prim's and Kruskal's algorithms for finding MST

Concepts

A Minimum Spanning Tree of a connected graph is a subgraph that connects all the vertices together while minimizing the total edge weight. Prim's algorithm and Kruskal's algorithm are two popular algorithms used to find MST.

Lecture 3: Topological Sort

Learning Objectives

* Understand the concept of Topological Sort

* Learn how to implement a Topological Sort algorithm

Concepts

A Topological Sort is an ordering of the vertices in a directed acyclic graph (DAG) such that for every directed edge u -> v, vertex u comes before v in the ordering. Topological Sort is used in applications such as scheduling tasks and ordering dependencies.

Lab 1: Implementing Graph Traversal Algorithms

* Implement DFS and BFS algorithms in a programming language of your choice

* Test your implementation with sample graphs

* Compare the time and space complexity of DFS and BFS algorithms

Week 11

Week 11: Graph Algorithms

Lecture 1: Graph Traversal Algorithms (3 hours)

Learning Objectives

• Understand the different types of graph traversal algorithms

• Learn how to implement Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms

• Apply graph traversal algorithms to solve real-world problems

Concepts

Graph traversal algorithms are used to traverse and search through a graph. There are two primary types: Breadth-First Search (BFS) and Depth-First Search (DFS). BFS explores all the nodes at a given depth before moving on to the next depth level, while DFS explores as far as possible along each branch before backtracking. We will implement both BFS and DFS algorithms using adjacency lists and adjacency matrices.

Lecture 2: Graph Traversal Algorithms (continued)

Learning Objectives

• Understand the time and space complexity of BFS and DFS algorithms

• Learn how to implement DFS with recursion and iteration

• Compare and contrast BFS and DFS algorithms

Concepts

The time complexity of BFS is O(V + E) since we visit each node and edge once. The space complexity is O(V) due to the use of a queue to store nodes. DFS has a time complexity of O(V + E) as well, but the space complexity is O(h) where h is the maximum depth of the recursion. We will also explore the use of recursion and iteration in implementing DFS.

Lecture 3: Graph Traversal Algorithms (continued)

Learning Objectives

• Understand how to optimize graph traversal algorithms for large graphs

• Learn how to implement topological sorting using DFS

• Apply graph traversal algorithms to solve real-world problems

Concepts

For large graphs, we can optimize graph traversal algorithms by using techniques such as memoization and caching. We will also learn how to implement topological sorting using DFS, which is used to order the vertices of a directed acyclic graph (DAG) such that for every edge (u,v),

vertex u comes before v in the ordering.

Lab 1: Implementing Graph Traversal Algorithms (1 hour)

Learning Objectives

• Implement BFS and DFS algorithms using adjacency lists and adjacency matrices

• Apply graph traversal algorithms to solve real-world problems

• Compare and contrast the performance of BFS and DFS algorithms

Concepts

In this lab, we will implement BFS and DFS algorithms using adjacency lists and adjacency matrices. We will also apply graph traversal algorithms to solve real-world problems and compare and contrast the performance of BFS and DFS algorithms.

Week 12

Week 12: Greedy Algorithms and Dynamic Programming

Lecture 1: Introduction to Greedy Algorithms (30 minutes)

Greedy Algorithms

Greedy algorithms are a type of algorithm that make the optimal choice at each step as it attempts to find the overall optimal solution.

Learning Objectives

* Define greedy algorithms and their applications

* Identify when to use greedy algorithms

Concepts

* Greedy choice property

* Examples: Huffman coding, activity selection problem

Lecture 2: Examples of Greedy Algorithms (30 minutes)

Greedy Algorithms Examples

We will discuss two classic examples of greedy algorithms: Huffman coding and activity selection problem.

Learning Objectives

* Apply greedy algorithms to real-world problems

* Analyze the time and space complexity of greedy algorithms

Concepts

* Huffman coding algorithm

* Activity selection problem solution

Lecture 3: Introduction to Dynamic Programming (30 minutes)

Dynamic Programming

Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems.

Learning Objectives

* Define dynamic programming and its applications

* Identify when to use dynamic programming

Concepts

* Overlapping subproblems

* Memoization

Lab 1: Implementing Greedy Algorithms and Dynamic Programming (60 minutes)

* Implement the Huffman coding algorithm using a greedy approach

* Solve the activity selection problem using dynamic programming

* Analyze the time and space complexity of the implemented algorithms

Week 13

Week 13: Dynamic Programming

3-0

Lecture 1: Dynamic Programming Introduction

Dynamic Programming Introduction

Learning Objectives

* Define dynamic programming and its applications

* Identify the characteristics of dynamic programming problems

Concepts

Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems. It involves constructing a solution to a given problem by combining solutions to its subproblems. Dynamic programming is used to solve problems that have the following characteristics: optimal substructure, overlapping subproblems, and a way to store and reuse solutions to subproblems.

Lecture 2: Dynamic Programming Example

Dynamic Programming Example

Learning Objectives

* Understand the concept of optimal substructure

* Apply dynamic programming to solve a problem

Concepts

The Fibonacci sequence is a classic example of a dynamic programming problem. The sequence is defined as: F(n) = F(n-1) + F(n-2). To solve this problem using dynamic programming, we can create a table to store the solutions to subproblems and then use these solutions to compute the final answer.

Lecture 3: Dynamic Programming with Memoization

Dynamic Programming with Memoization

Learning Objectives

* Understand the concept of memoization

* Apply memoization to improve the efficiency of dynamic programming algorithms

Concepts

Memoization is a technique used in dynamic programming to store the solutions to subproblems in a table and reuse them to avoid redundant computations. This technique can significantly improve the efficiency of dynamic programming algorithms by reducing the time complexity from exponential to polynomial.

No Lab for Week 13

This concludes Week 13 of the course. In the next week, we will cover more advanced topics in dynamic programming.

Week 14

Week 14: Dynamic Programming

Lecture 1: Introduction to Dynamic Programming (≈150 words)

Dynamic Programming Basics

Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems.

Learning Objectives

• Understand the basic concept of dynamic programming

• Identify problems that can be solved using dynamic programming

Concepts

• Overlapping Subproblems

• Optimal Substructure

• Memoization

Lecture 2: Example Problems (≈150 words)

Fibonacci Sequence

The Fibonacci sequence is a classic example of a problem that can be solved using dynamic programming.

Learning Objectives

• Apply dynamic programming to solve the Fibonacci sequence problem

• Understand the role of memoization in dynamic programming

Concepts

• Fibonacci sequence

• Memoization technique

• Bottom-up approach

Lecture 3: Matrix Chain Multiplication (≈150 words)

Matrix Chain Multiplication

The matrix chain multiplication problem is another example of a problem that can be solved using dynamic programming.

Learning Objectives

• Apply dynamic programming to solve the matrix chain multiplication problem

• Understand the concept of optimal substructure

Concepts

• Matrix chain multiplication

• Optimal substructure

• Dynamic programming approach

Lab 1: Implement Dynamic Programming (≈150 words)

Implementing Dynamic Programming

Implement the dynamic programming approach to solve the Fibonacci sequence and matrix chain multiplication problems.

Tasks

• Write a Python program to solve the Fibonacci sequence problem using dynamic programming

• Write a Python program to solve the matrix chain multiplication problem using dynamic programming

Format

• Provide a clear and concise code

• Use comments to explain the code

• Test the code with sample inputs

Week 15

Week 15: Sorting Algorithms (3-1)

Lecture 1: Introduction to Sorting Algorithms (1.5 hours)

Sorting Algorithms

Learning Objectives

• Define sorting algorithms and their importance in data structures.

• Identify common sorting algorithms.

Concepts

Sorting algorithms rearrange data elements in a specific order. Sorting is essential in various applications, including databases, web search engines, and data analysis. Common sorting algorithms include Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.

Lecture 2: Bubble Sort and Selection Sort (1.5 hours)

Bubble Sort and Selection Sort

Learning Objectives

• Implement Bubble Sort and Selection Sort algorithms.

• Analyze the time and space complexity of Bubble Sort and Selection Sort.

Concepts

Bubble Sort: Repeatedly iterates through the data, swapping adjacent elements if they are in the wrong order. Time complexity: O(n^2). Space complexity: O(1).

Selection Sort: Repeatedly selects the smallest element from the unsorted portion of the data and swaps it with the first unsorted element. Time complexity: O(n^2). Space complexity: O(1).

Lecture 3: Insertion Sort and Merge Sort (1 hour)

Insertion Sort and Merge Sort

Learning Objectives

• Implement Insertion Sort and Merge Sort algorithms.

• Analyze the time and space complexity of Insertion Sort and Merge Sort.

Concepts

Insertion Sort: Builds the sorted array one element at a time by inserting each element into its proper position. Time complexity: O(n^2) in the worst case. Space complexity: O(1).

Merge Sort: Divides the array into two halves, sorts each half recursively, and merges the sorted halves. Time complexity: O(n log n). Space complexity: O(n).

Lab 1: Implementing Sorting Algorithms (1 hour)

Lab 1: Sorting Algorithms Implementation

Learning Objectives

• Implement the sorting algorithms discussed in the lectures.

• Compare the performance of different sorting algorithms.

Concepts

Implement the Bubble Sort, Selection Sort, Insertion Sort, and Merge Sort algorithms in a programming language of your choice. Compare the time complexity of each algorithm using a large dataset.

Week 16

Week 16: Advanced Graph Algorithms and Data Structures

3-1 (3 lectures, 1 lab)

Lecture 1: Topological Sorting

Learning Objectives

• Understand the concept of topological sorting

• Learn how to implement topological sorting using DFS and Kahn's algorithm

• Apply topological sorting to real-world scenarios

Concepts

Topological sorting is a linear ordering of vertices in a directed acyclic graph (DAG) such that for every directed edge u -> v, vertex u comes before v in the ordering. Topological sorting has applications in scheduling tasks with dependencies and ordering operations in a database.

Lecture 2: Minimum Spanning Tree

Learning Objectives

• Understand the concept of minimum spanning tree

• Learn how to implement Kruskal's and Prim's algorithms for finding the minimum spanning tree

• Apply minimum spanning tree to real-world network optimization problems

Concepts

A minimum spanning tree of a connected weighted graph is a subset of the edges that connect all the vertices together while minimizing the total edge weight. Kruskal's and Prim's algorithms are popular methods for finding the minimum spanning tree.

Lecture 3: Advanced Graph Algorithms

Learning Objectives

• Understand the concept of graph diameters and centralities

• Learn how to calculate graph diameters and centralities using various algorithms

• Apply graph diameters and centralities to real-world network analysis

Concepts

Graph diameters and centralities are important metrics for understanding the structure and properties of a graph. They can be used to identify key nodes and edges in a network, and to analyze the overall connectivity and robustness of the graph.

Lab 1: Implementing Topological Sorting and Minimum Spanning Tree

• Implement topological sorting using DFS and Kahn's algorithm

• Implement Kruskal's and Prim's algorithms for finding the minimum spanning tree

• Apply the algorithms to real-world datasets and visualize the results